

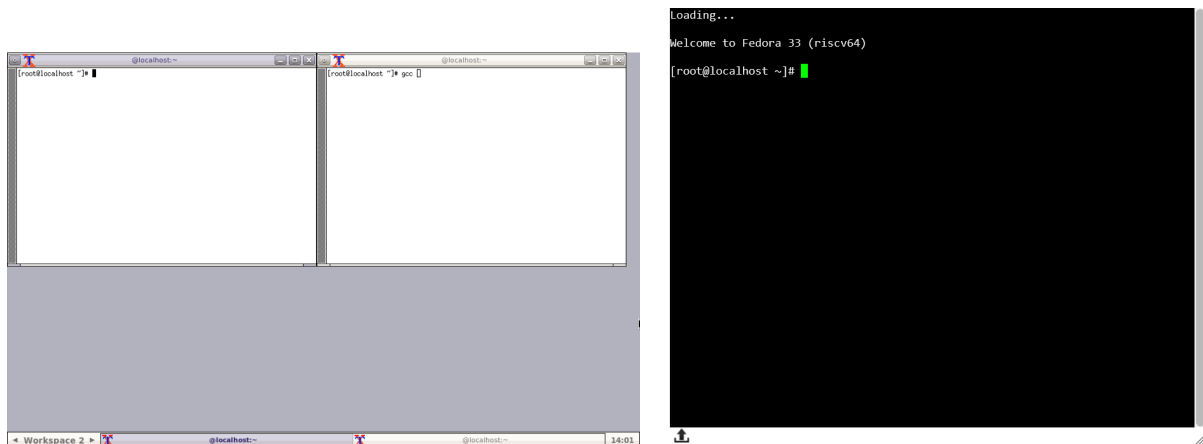
Session 8: Users, Groups, Permissions

On completion of these activities you should be able to:

- Use the `ls` command to list and understand information about file names, in particular their access permissions.
- Create new users and groups and add users to groups.
- Modify file permissions to implement a chosen access policy.
- Set the `setuid` bit on a file and know what it does

Task 0

Start a unix environment in JSLinux (<https://bellard.org/jslinux/>). You can select the Fedora X-Windows environment or the Console.



Task 1

As user `root`, run the command `ls -la` in your home directory. The `-l` option chooses a "long listing" (including access permission information, etc). The `-a` option means "show all files" (by default, file names starting with a dot are treated as "hidden"). You should get a response similar to this:

```
[root@localhost ~]# ls -la
total 40
dr-xr-x---  2 root root 243 Dec 26  2020 .
drwxrwxrwx 21 root root 550 Mar 13 15:14 ..
-rw-r--r--  1 root root  18 Jul 29  2020 .bash_logout
-rw-r--r--  1 root root 141 Jul 29  2020 .bash_profile
-rw-r--r--  1 root root 429 Jul 29  2020 .bashrc
-rw-r--r--  1 root root 114 Dec 26  2020 bench.py
-rw-r--r--  1 root root 100 Jul 29  2020 .cshrc
-rw-r--r--  1 root root 106 Dec 19  2020 .fldev_cfg
-rw-r--r--  1 root root 185 Sep  9  2018 hello.c
-rw-r--r--  1 root root 129 Jul 29  2020 .tcshrc
[root@localhost ~]#
```

Recall the meaning of the permission strings (Session 07, see also en.wikipedia.org/wiki/File_system_permissions). Identify the files which `root` can read/write/execute but other users cannot. Which of these files are directories?

Task 2

As user `root`, use the `mkdir` command to create a new directory `session08`. Use an editor (or the `touch` command) to create four new files inside `session08`, called:

`file1`, `file2`, `file3`, `file4`.

Inside `session08`, run `ls -l`. The output should look something like this:

```
[root@localhost session08]# ls -la
total 8
drwxr-xr-x 2 root root 125 Mar 13 15:28 .
dr-xr-x-- 3 root root 269 Dec 26 2020 ..
-rw-r--r-- 1 root root  0 Mar 13 15:28 file1
-rw-r--r-- 1 root root  0 Mar 13 15:28 file2
-rw-r--r-- 1 root root  0 Mar 13 15:28 file3
-rw-r--r-- 1 root root  0 Mar 13 15:28 file4
```

tou

Task 3

The commands below can be used to create new users and new groups and to add users to groups; they can only be run by `root`:

adduser *new-user-name* creates a new user with the given name

groupadd *new-group-name* creates a new group with the given name (sometimes is used as **addgroup**)

adduser *user-name -g group-name* adds the named user to the named group

Use the above commands to create new users and groups and to add users to groups, as follows:

a) create groups **geek** and **cobbler**

b) create users **troy** and **zapata** in group **geek**

d) create user **maryam** in group **cobbler**

Change from `root` to user `troy` with the command `su troy` and create `file5` using `touch` as previously done. What happens? Is this what you were expecting?

Try to open `file1` with `vi` or other editors, what happens? (if you cannot exit `vi` with `Esc - zz`, try `:qa!`)

Task 4

The reason you were not able to create a file is due to the permissions of the current directory, i.e. the single dot in the previous figure. Return to `root` (use the `exit` command and verify with `whoami`), and change the permissions of the directory to `777` with

```
chmod 777 .
```

Remember that a single dot (`.`) in unix means “here” and two dots (`..`) mean “the directory above”. Now the list should like this:

```
[root@localhost session08]# ls -la
total 8
drwxrwxrwx 2 root root 125 Mar 13 15:20 .
dr-xr-x--- 3 root root 269 Dec 26 2020 ..
-rw-r--r-- 1 root root  0 Mar 13 15:23 file1
-rw-r--r-- 1 root root  0 Mar 13 15:23 file2
-rw-r--r-- 1 root root  0 Mar 13 15:23 file3
-rw-r--r-- 1 root root  0 Mar 13 15:23 file4
[root@localhost session08]#
```

Now, try again to create files as the different users that you previously created. Use `troy` for `file5`, `zapata` for `file6` and `maryam` for `file7`.

Your listings should like this:

```
[maryam@localhost session08]$ ls -la
total 8
drwxrwxrwx 2 root root 191 Mar 13 15:28 .
dr-xr-x--- 3 root root 269 Dec 26 2020 ..
-rw-r--r-- 1 root root  0 Mar 13 15:28 file1
-rw-r--r-- 1 root root  0 Mar 13 15:28 file2
-rw-r--r-- 1 root root  0 Mar 13 15:28 file3
-rw-r--r-- 1 root root  0 Mar 13 15:28 file4
-rw-r--r-- 1 troy geek  0 Mar 13 15:35 file5
-rw-r--r-- 1 zapata geek  0 Mar 13 15:35 file6
-rw-r--r-- 1 maryam cobbler  0 Mar 13 15:37 file7
```

Task 5

The owner of a list of files can be changed (by root) with the `chown` command:

```
chown user-name file-names...
```

The group owner of a list of files can be changed (by root) with the `chgrp` command:

```
chgrp group-name file-names...
```

- Change the owner of `file2` and `file3` to `maryam`.
- Change the group owner of `file1` and `file3` to `geek`.

c) Change the group owner of `file2` and `file4` to `cobbler`.

Once again, run `ls -la`. The output should now look like this:

```
[root@localhost session08]# ls -la
total 8
drwxrwxrwx 2 root    root    191 Mar 13 15:28 .
dr-xr-x--- 3 root    root    269 Dec 26 2020 ..
-rw-r--r-- 1 root    geek     0 Mar 13 15:28 file1
-rw-r--r-- 1 maryam  cobbler   0 Mar 13 15:28 file2
-rw-r--r-- 1 maryam  geek     0 Mar 13 15:28 file3
-rw-r--r-- 1 root    cobbler   0 Mar 13 15:28 file4
-rw-r--r-- 1 troy    geek     0 Mar 13 15:35 file5
-rw-r--r-- 1 zapata  geek     0 Mar 13 15:35 file6
-rw-r--r-- 1 maryam  cobbler   0 Mar 13 15:37 file7
```

Task 6

There are two modes for setting permissions on a file: *symbolic* and *numeric*. The symbolic mode allows you easily to change permissions for one category of user while leaving other parts unchanged; the numeric mode requires you to specify all parts at once. I generally use the symbolic mode but, if bit-twiddling is your thing, you may prefer the numeric mode when it is applicable. To change permissions on a file, use the `chmod` command. The basic (symbolic) uses of this command take the form

```
chmod who+what file-name
```

```
chmod who-what filename
```

where *who* can be either **u** (user=owner) **g** (group) **o** (other) or **a** (all), and *what* can be either **r** (read) **w** (write) or **x** (execute). So, for example, to give the owner of file `thingOne` permission to execute it, we could use:

```
chmod u+x thingOne
```

And to prevent file `thingOne`'s group owners from writing to it we could use:

```
chmod g-w thingOne
```

Alternatively, we could set the owner, group and other permissions to `rw-`, `r--`, and `r--` (respectively) in one go, using numeric mode:

```
chmod 644 thingOne
```

Suppose there is an additional user `chawla` who does **not** belong to either group `geek` or group `cobbler`. Modify the access permissions on `file1`, `file2`, `file3`, `file4` so that the access matrix for users `troy`, `zapata`, `maryam` and `chawla` looks like this:

	file1	file2	file3	file4
troy	r w	r	r	r w x
maryam	r	r w x	r w	r w
zapata	r w	r w x	r	r w
chawla	r	r		r

Task 7

These three commands are about the `setuid` feature of UNIX file systems. In addition to the file-permission bits controlling which users can read, write and execute a file, there is a group of bits which include the so-called “setuid” bit. If the setuid bit is set on a file and the file is executable, the process created when it is run will execute with the file *owner’s uid* as its “effective” uid, rather than the uid of the user who actually runs it. For instance, we will look at the following three permissions. Run

```
ls -l /bin/ls
```

```
ls -l /bin/ls
```

```
ls -l /bin/ls
```

Your output should look something like this:

```
[troy@localhost homeTroy]$ ls -l /bin/ls
-rwxr-xr-x 1 root root 127152 Jul 13  2020 /bin/ls
[troy@localhost homeTroy]$ ls -l /bin/sudo
---s---x--x 1 root root 150016 Nov 26  2020 /bin/sudo
[troy@localhost homeTroy]$ ls -l /bin/passwd
-rwsr-xr-x 1 root root 24760 Aug 14  2020 /bin/passwd
[troy@localhost homeTroy]$
```

Notice the difference between the permissions. First, `ls` allows the owner, group and everyone else to read and execute the command, but only the owner to write to it. The command `sudo`, which allows to run commands as root only allows execution and it has an “s” instead of an “x” for the owner. This will allow to be run with `root` privileges. A similar case is the command `passwd` that allows users to change their password and thus requires `root` privileges to access the file where the passwords are saved.

Task 8 [optional]

Find out how to set a password for a user that you create and how to change it. Find out where the passwords are saved and try to access the file.